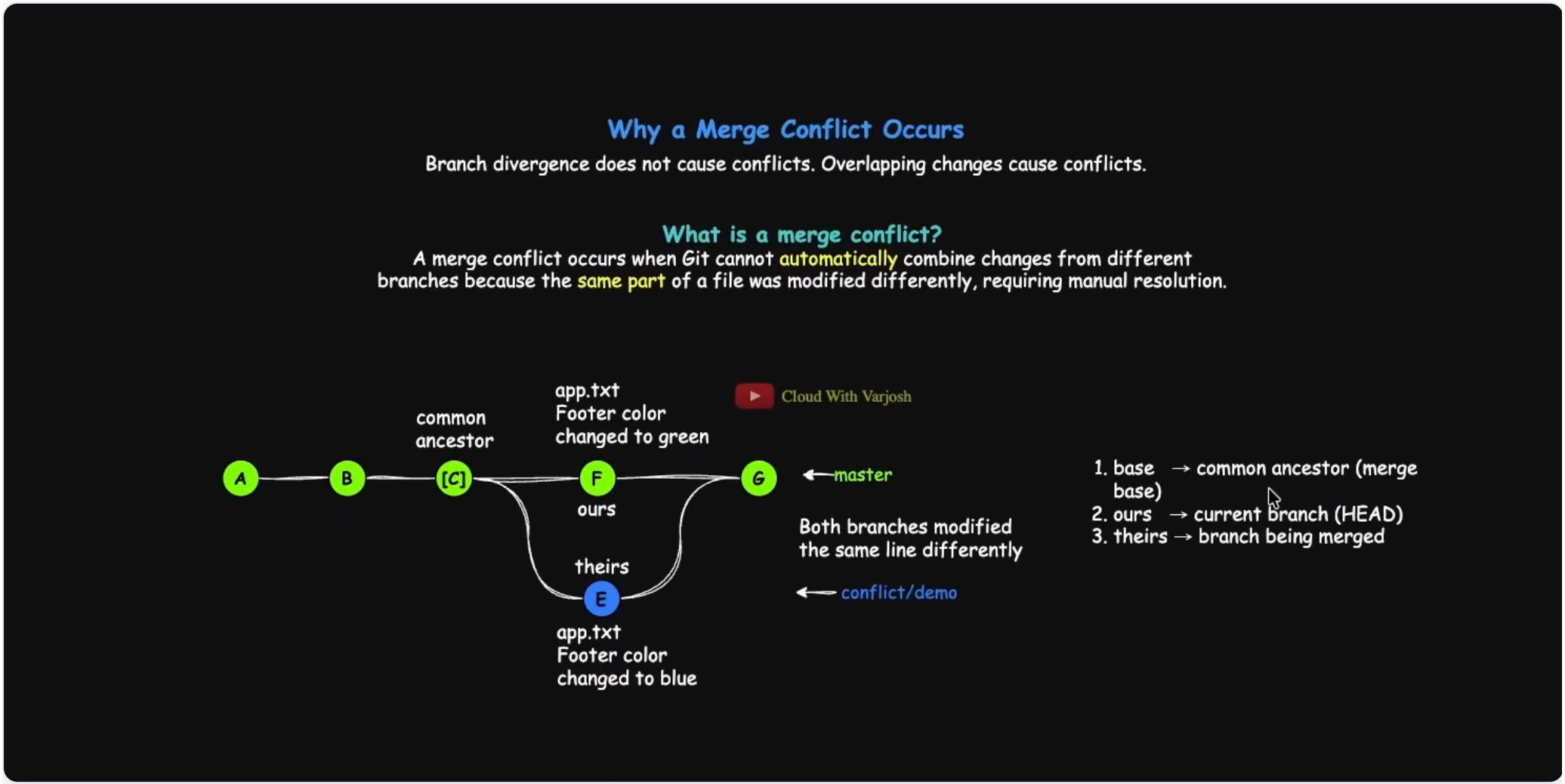
GIT TRACK · SCREENSHOTS

Chapter 03 — Merge Conflicts & Undoing Commits
Conflict Markers, Reset (soft / mixed / hard) & Revert

Git Masterclass · Part 1 · stills from the session, in order · 2026-10-13

26 screenshots


Why a Merge Conflict Occurs
Branch divergence does not cause conflicts. Overlapping changes cause conflicts.
What is a merge conflict?
A merge conflict occurs when Git cannot automatically combine changes from different branches because the same part of a file was modified differently, requiring manual resolution.
Cloud With Varjosh
common ancestor
app.txt Footer color changed to green
A
B
[C]
F
G
master
ours
Both branches modified the same line differently
theirs
E
conflict/demo
app.txt Footer color changed to blue
1. base → common ancestor (merge base)
2. ours → current branch (HEAD)
3. theirs → branch being merged

02 / 26 Starting point: master at b18b077

```
root@cka-w2 ~/project-files (master)
$ git log --oneline
b18b077 (HEAD -> master) merge branch bugfix/typo
d40c83f style: improve footer spacing
3fc7beb fix: correct footer typo
13b779a (feature/login) feat: implement login functionality
9aaeab0 feat: add footer section
fcaef20 style: improve application header styling
0e3ffcd chore: initialize application
root@cka-w2 ~/project-files (master)
$
```

git checkout -b conflict/demo

```
$ git log --oneline
b18b077 (HEAD -> master) merge branch bugfix/typo
d40c83f style: improve footer spacing
3fc7beb fix: correct footer typo
13b779a (feature/login) feat: implement login functionality
9aaeab0 feat: add footer section
fcaef20 style: improve application header styling
0e3ffcd chore: initialize application
root@cka-w2 ~/project-files (master)
$ git checkout -b conflict/demo
Switched to a new branch 'conflict/demo'
root@cka-w2 ~/project-files (conflict/demo)
$ ls
app.txt  bufix.txt  layout.txt  login
root@cka-w2 ~/project-files (conflict/demo)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
root@cka-w2 ~/project-files (conflict/demo)
```

conflict/demo: footer color blue (272dbc9)

```
root@cka-w2 ~/project-files (conflict/demo)
$ echo "footer color changed to blue" >> app.txt
root@cka-w2 ~/project-files (conflict/demo)
$ git add app.txt
root@cka-w2 ~/project-files (conflict/demo)
$ git commit -m "style: change footer color to blue"
[conflict/demo 272dbc9] style: change footer color to blue
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (conflict/demo)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
footer color changed to blue
root@cka-w2 ~/project-files (conflict/demo)
```

master: footer colour green (7900065)

```
root@cka-w2 ~/project-files (conflict/demo)
$ git checkout master
Switched to branch 'master'
root@cka-w2 ~/project-files (master)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
root@cka-w2 ~/project-files (master)
$ echo "footer color changed to green" >> app.txt
root@cka-w2 ~/project-files (master)
$ git add app.txt
root@cka-w2 ~/project-files (master)
$ git commit -m "style: change footer colour to green"
[master 7900065] style: change footer colour to green
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (master)
```

app.txt on master

```
root@cka-w2 ~/project-files (master)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
root@cka-w2 ~/project-files (master)
$ echo "footer color changed to green" >> app.txt
root@cka-w2 ~/project-files (master)
$ git add app.txt
root@cka-w2 ~/project-files (master)
$ git commit -m "style: change footer colour to green"
[master 7900065] style: change footer colour to green
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (master)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
footer color changed to green
root@cka-w2 ~/project-files (master)
$
```

CONFLICT (content): Merge conflict in app.txt

```
root@cka-w2 ~/project-files (master)
$ git merge conflict/demo
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.
root@cka-w2 ~/project-files (master)
$
```

Conflict markers in app.txt

```
root@cka-w2 ~/project-files (master)
$ ls
app.txt  bufix.txt  layout.txt  login
root@cka-w2 ~/project-files (master)
$ ls -l app.txt
-rw-r--r-- 1 root root 196 Oct 13 12:49 app.txt
root@cka-w2 ~/project-files (master)
$ cat app.txt
application initialized
updated aplication header styling
Added footer section to application
<<<<<<< HEAD
footer color changed to green
=======
footer color changed to blue
>>>>>>> conflict/demo
root@cka-w2 ~/project-files (master)
$
```

git add + git commit → merge commit 09ba6f9

```
root@cka-w2 ~/project-files (master)
$ git statusgit add app.txt
git: 'statusgit' is not a git command. See 'git --help'.
root@cka-w2 ~/project-files (master)
$ git add app.txt
root@cka-w2 ~/project-files (master)
$ git commit
[master 09ba6f9] Merge branch 'conflict/demo'
root@cka-w2 ~/project-files (master)
$ git commit -m "resolve conflict"
On branch master
nothing to commit, working tree clean
root@cka-w2 ~/project-files (master)
$
```

10 / 26 git log --oneline after the merge

```
root@cka-w2 ~/project-files (master)
$ git log --oneline
09ba6f9 (HEAD -> master) Merge branch 'conflict/demo'
7900065 style: change footer colour to green
272dbc9 (conflict/demo) style: change footer color to blue
b18b077 merge branch bugfix/typo
d40c83f style: improve footer spacing
3fc7beb fix: correct footer typo
13b779a (feature/login) feat: implement login functionality
9aaeab0 feat: add footer section
fcaef20 style: improve application header styling
0e3ffcd chore: initialize application
root@cka-w2 ~/project-files (master)
$
```

git log --oneline --graph

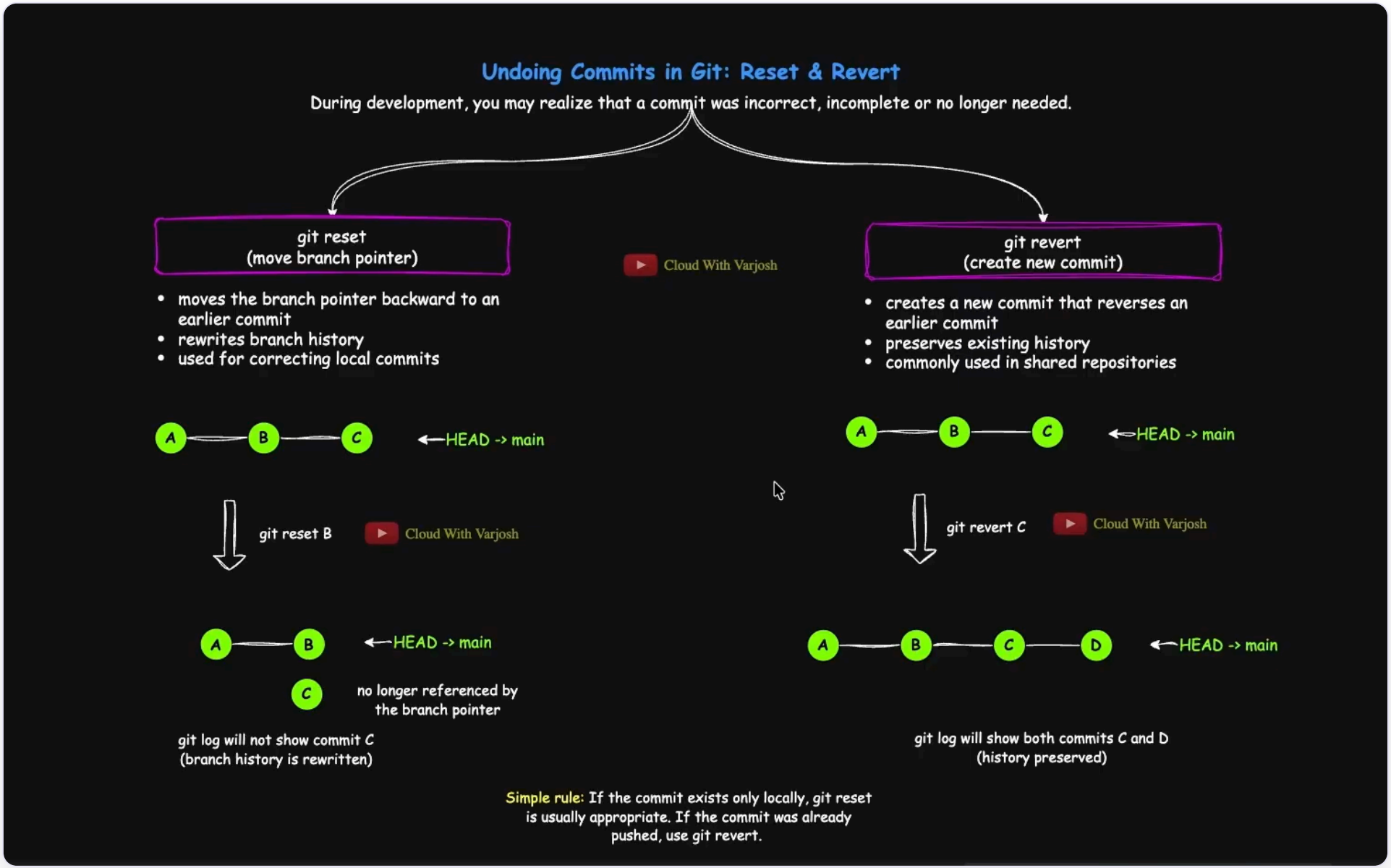
```
root@cka-w2 ~/project-files (master)
$ git log --oneline --graph
*   09ba6f9 (HEAD -> master) Merge branch 'conflict/demo'
|\
| * 272dbc9 (conflict/demo) style: change footer color to blue
* | 7900065 style: change footer colour to green
|/
*   b18b077 merge branch bugfix/typo
|\
| * 3fc7beb fix: correct footer typo
* | d40c83f style: improve footer spacing
|/
* 13b779a (feature/login) feat: implement login functionality
* 9aaeab0 feat: add footer section
* fcaef20 style: improve application header styling
* 0e3ffcd chore: initialize application
root@cka-w2 ~/project-files (master)
$
```

git reflog — commit (merge)

```
root@cka-w2 ~/project-files (master)
$ git reflog
09ba6f9 (HEAD -> master) HEAD@{0}: commit (merge): Merge branch 'conflict/demo'
7900065 HEAD@{1}: commit: style: change footer colour to green
b18b077 HEAD@{2}: checkout: moving from conflict/demo to master
272dbc9 HEAD@{3}: commit: style: change footer color to blue
b18b077 HEAD@{4}: checkout: moving from master to conflict/demo
b18b077 HEAD@{5}: merge bugfix/typo: Merge made by the 'ort' strategy.
d40c83f HEAD@{6}: checkout: moving from bugfix/typo to master
3fc7beb HEAD@{7}: checkout: moving from master to bugfix/typo
d40c83f HEAD@{8}: commit: style: improve footer spacing
13b779a (feature/login) HEAD@{9}: checkout: moving from bugfix/typo to master
3fc7beb HEAD@{10}: commit: fix: correct footer typo
13b779a (feature/login) HEAD@{11}: checkout: moving from master to bugfix/typo
13b779a (feature/login) HEAD@{12}: checkout: moving from bugfix/typo to master
0e3ffcd HEAD@{13}: checkout: moving from master to bugfix/typo
13b779a (feature/login) HEAD@{14}: checkout: moving from bugfix/typo to master
13b779a (feature/login) HEAD@{15}: checkout: moving from master to bugfix/typo
13b779a (feature/login) HEAD@{16}: merge feature/login: Fast-forward
9aaeab0 HEAD@{17}: checkout: moving from feature/login to master
13b779a (feature/login) HEAD@{18}: commit: feat: implement login functionality
9aaeab0 HEAD@{19}: checkout: moving from master to feature/login
9aaeab0 HEAD@{20}: checkout: moving from feature/login to master
9aaeab0 HEAD@{21}: checkout: moving from master to feature/login
9aaeab0 HEAD@{22}: checkout: moving from feature/login to master
9aaeab0 HEAD@{23}: checkout: moving from master to feature/login
9aaeab0 HEAD@{24}: commit: feat: add footer section
fcaef20 HEAD@{25}: commit: style: improve application header styling
0e3ffcd HEAD@{26}: commit (initial): chore: initialize application
root@cka-w2 ~/project-files (master)
```

13 / 26 git branch -m master main

```
root@cka-w2 ~/project-files (master)
$ git branch -m master main
root@cka-w2 ~/project-files (main)
$
```


Undoing Commits in Git: Reset & Revert
During development, you may realize that a commit was incorrect, incomplete or no longer needed.
git reset
(move branch pointer)
Cloud With Varjosh
git revert
(create new commit)
moves the branch pointer backward to an earlier commit
rewrites branch history
used for correcting local commits
creates a new commit that reverses an earlier commit
preserves existing history
commonly used in shared repositories
A
B
C
HEAD -> main
git reset B
Cloud With Varjosh
git revert C
Cloud With Varjosh
D
no longer referenced by the branch pointer
git log will not show commit C (branch history is rewritten)
git log will show both commits C and D (history preserved)
Simple rule: If the commit exists only locally, git reset is usually appropriate. If the commit was already pushed, use git revert.

Three commits: feature A, B, c

```
Initialized empty Git repository in /root/project-files/.git/
root@cka-w2 ~/project-files
$ echo "farture A" > app.txt
root@cka-w2 ~/project-files
$ git add .
root@cka-w2 ~/project-files
$ git commit -m "feat: add feature A"
[master (root-commit) 434db70] feat: add feature A
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt
root@cka-w2 ~/project-files (master)
$ echo "farture B" >> app.txt
root@cka-w2 ~/project-files (master)
$ git add .
root@cka-w2 ~/project-files (master)
$ git commit -m "feat: add feature B"
[master d516faa] feat: add feature B
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (master)
$ echo "farture c" >> app.txt
root@cka-w2 ~/project-files (master)
$ git add .
root@cka-w2 ~/project-files (master)
$ git commit -m "feat: add feature c"
[master 522bf5f] feat: add feature c
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (master)
$
```

```
root@cka-w2 ~/project-files (master)
$ cat app.txt
farture A
farture B
farture c
root@cka-w2 ~/project-files (master)
$
```

git log --oneline

```
root@cka-w2 ~/project-files (master)
$ git log --oneline
522bf5f (HEAD -> master) feat: add feature c
d516faa feat: add feature B
434db70 feat: add feature A
root@cka-w2 ~/project-files (master)
$
```

git reset d516faa → unstaged changes

```
root@cka-w2 ~/project-files (master)
$ git reset d516faa
Unstaged changes after reset:
M       app.txt
root@cka-w2 ~/project-files (master)
$
```

c is gone from the branch

```
root@cka-w2 ~/project-files (master)
$ git reset d516faa
Unstaged changes after reset:
M       app.txt
root@cka-w2 ~/project-files (master)
$ git log --oneline
d516faa (HEAD -> master) feat: add feature B
434db70 feat: add feature A
root@cka-w2 ~/project-files (master)
$
```

20 / 26 git status: app.txt modified

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")
root@cka-w2 ~/project-files (master)
$
```

21 / 26 Edit app.txt

```
root@cka-w2 ~/project-files (master)
$ vim app.txt
root@cka-w2 ~/project-files (master)
$ cat app.txt
farture A
farture B
farture c added few more changes
root@cka-w2 ~/project-files (master)
$
```

Recommit 7b015bd

```
root@cka-w2 ~/project-files (master)
$ vim app.txt
root@cka-w2 ~/project-files (master)
$ cat app.txt
farture A
farture B
farture c added few more changes
root@cka-w2 ~/project-files (master)
$ git add app.txt
root@cka-w2 ~/project-files (master)
$ git commit -m "feat: made few changes"
[master 7b015bd] feat: made few changes
 1 file changed, 1 insertion(+)
root@cka-w2 ~/project-files (master)
$ git status
On branch master
nothing to commit, working tree clean
root@cka-w2 ~/project-files (master)
$ git log --oneline
7b015bd (HEAD -> master) feat: made few changes
d516faa feat: add feature B
434db70 feat: add feature A
root@cka-w2 ~/project-files (master)
$
```

23 / 26 git reflog — 522bf5f still recorded

```
root@cka-w2 ~/project-files (master)
$ git reflog
7b015bd (HEAD -> master) HEAD@{0}: commit: feat: made few changes
d516faa HEAD@{1}: reset: moving to d516faa
522bf5f HEAD@{2}: commit: feat: add feature c
d516faa HEAD@{3}: commit: feat: add feature B
434db70 HEAD@{4}: commit (initial): feat: add feature A
root@cka-w2 ~/project-files (master)
$
```

Types of Reset

`git reset` behaves differently depending on the **mode used**. Each mode determines what happens to the **commit history, the staging area, and the working directory**.

Assume the following commit history:

```
A --- B --- C   (HEAD -> main)
```

Commit **C** added a new line to `app.txt` :

```
Login feature added
```

Now we run:

```
git reset B
```

The behavior depends on the reset mode.

Reset Type	What Happens	Example (`app.txt`)	Typical Next Action
`--soft`	branch moves back; **changes remain staged**	commit C is no longer in branch history, but `Login feature added` is still staged	usually **recommit immediately** (possibly with a corrected commit message)
`--mixed` (default)	branch moves back; **changes become unstaged**	commit C is no longer in branch history, but the line remains as an unstaged change	**edit if needed → `git add` → comm again**
`--hard`	branch moves back; **changes are removed from working directory**	commit C is no longer in branch history and the line is removed from `app.txt`	**start fresh → make changes → s → commit again**

> **Note:** The commit is not immediately deleted. It becomes **unreachable from the branch** and can often be recovered using `git reflog`.

Simple rule: local → reset, pushed → revert

